



Formation 3 Jours — Jour 2

Modules 3 & 4 · Architecture d'application + Interface utilisateur



Sommaire — Jour 2

01 State & Props — Les fondamentaux

03 Le pattern Flux

05 Alternatives : Zustand, Jotai, TanStack Query

07 Flexbox & Styling

09 Composants avancés & communauté

11 TP — Initialisation projet & première app

02 React Native & MVC

04 Redux & Redux Toolkit

06 Composants de base & cycles de vie

08 Navigation (Expo Router & React Navigation)

10 Animations & transitions

Module 3 · Architecture d'application

State, Props, MVC, Flux & Redux



Les Props — Données en lecture seule

```
// Props : données passées par le parent – immuables dans l'enfant
interface CarteProps {
  titre: string;
  note: number;
  couleur?: string;      // prop optionnelle
  onPress: (id: string) => void;
}

const Carte: React.FC<CarteProps> = ({
  titre,
  note,
  couleur = '#0d47a1',   // valeur par défaut
  onPress,
}) => (
  <TouchableOpacity onPress={() => onPress(titre)}>
    <Text style={{ color: couleur }}>{titre} – ★ {note}</Text>
  </TouchableOpacity>
);
```



Le State — Données réactives

```
import { useState, useReducer } from 'react';

// useState – state simple
function Compteur() {
  const [count, setCount] = useState<number>(0);
  // Mise à jour fonctionnelle (si le nouvel état dépend de l'ancien)
  const inc = () => setCount(prev => prev + 1);
  return <Button title={` ${count} `} onPress={inc} />;
}

// ⚠ Règles immuabilité – NE JAMAIS muter directement
// ✗ user.age = 30;
// ✔ setUser(prev => ({ ...prev, age: 30 }));
// ✗ items.push(newItem);
// ✔ setItems(prev => [...prev, newItem]);
```



useReducer — State complexe

```
type Action =
  | { type: 'ADD_ITEM'; payload: string }
  | { type: 'REMOVE_ITEM'; payload: string }
  | { type: 'TOGGLE_DONE'; payload: string };

function reducer(state: Todo[], action: Action): Todo[] {
  switch (action.type) {
    case 'ADD_ITEM':
      return [...state, { id: Date.now().toString(), text: action.payload, done: false }];
    case 'REMOVE_ITEM':
      return state.filter(t => t.id !== action.payload);
    case 'TOGGLE_DONE':
      return state.map(t => t.id === action.payload ? { ...t, done: !t.done } : t);
    default:
      return state;
  }
}

const [todos, dispatch] = useReducer(reducer, []);
dispatch({ type: 'ADD_ITEM', payload: 'Apprendre React Native' });
```



Lifting State Up & Context API

```
// Lifting State : remonter le state au parent commun
function Parent() {
  const [valeur, setValeur] = useState('');
  return (
    <>
      <Saisie onChange={setValeur} />    {/* modifie */}
      <Apercu valeur={valeur} />        {/* consomme */}
    </>
  );
}

// Context API : state global sans bibliothèque
const ThemeCtx = createContext<'light' | 'dark'>('light');

function App() {
  const [theme, setTheme] = useState<'light' | 'dark'>('light');
  return (
    <ThemeCtx.Provider value={theme}>
      <Navigation />
    </ThemeCtx.Provider>
  );
}

// Consommer n'importe où dans l'arbre
const theme = useContext(ThemeCtx);
```



React Native & MVC

Le modèle MVC (Model-View-Controller) est natif en iOS/Android, mais **incompatible** avec le flux React :

```
MVC (bidirectionnel – difficile à déboguer sur grande app)
User → Controller → Model → View → User
```

```
React (unidirectionnel – prévisible)
Action → State → Render → View → Action
```

- MVC souffre du "Fat Controller" sur les grandes apps
- Le flux bidirectionnel rend le debugging complexe
- React impose un **flux unidirectionnel** qui s'intègre mal au MVC classique

Le Pattern Flux

- Créé par Meta en 2014 — réaction aux limites du MVC
- Données toujours dans un seul sens :

```
Action → Dispatcher → Store → View
  ↑_____|
  (interactions utilisateur)
```

- Action : objet `{ type: 'ADD_TODO', payload: { text: '...' } }`
- Dispatcher : centre de routage des actions
- Store : contient le state et la logique métier
- View : composants React qui affichent et déclenchent des actions

- 1. Source unique de vérité : tout le state de l'app dans un seul store
- 2. State en lecture seule : seules les actions peuvent déclencher un changement
- 3. Changements via fonctions pures : les reducers sont déterministes

Composant → dispatch(action) → Reducer (pur) → Nouveau State → Composant

Redux est particulièrement adapté aux grandes apps avec state complexe partagé entre de nombreux composants éloignés dans l'arbre.

Redux Toolkit — Setup

```
npm install @reduxjs/toolkit react-redux
```

```
// store/todosSlice.ts – createSlice = actions + reducer en un
import { createSlice, PayloadAction } from '@reduxjs/toolkit';

const todosSlice = createSlice({
  name: 'todos',
  initialState: [] as Todo[],
  reducers: {
    // Immer intégré : mutations directes autorisées dans les reducers RTK
    ajouter: (state, action: PayloadAction<string>) => {
      state.push({ id: Date.now().toString(), text: action.payload, done: false });
    },
    supprimer: (state, action: PayloadAction<string>) => {
      return state.filter(t => t.id !== action.payload);
    },
    basculer: (state, action: PayloadAction<string>) => {
      const todo = state.find(t => t.id === action.payload);
      if (todo) todo.done = !todo.done; // Mutation directe OK (Immer)
    },
  },
});
export const { ajouter, supprimer, basculer } = todosSlice.actions;
export default todosSlice.reducer;
```



Configurer le Store & Connecter à l'app

```
// store/index.ts
import { configureStore } from '@reduxjs/toolkit';
import todosReducer from './todosSlice';

export const store = configureStore({
  reducer: { todos: todosReducer },
});

export type RootState = ReturnType<typeof store.getState>;
export type AppDispatch = typeof store.dispatch;
export const useAppSelector = useSelector.withTypes<RootState>();
export const useAppDispatch = useDispatch.withTypes<AppDispatch>();
```

```
// App.tsx – fournir le store avec Provider
import { Provider } from 'react-redux';
import { store } from './store';

export default function App() {
  return <Provider store={store}><Navigation /></Provider>;
}
```



RTK Query — Données async intégrées à Redux

```
// store/apiSlice.ts
import { createApi, fetchBaseQuery } from '@reduxjs/toolkit/query/react';

export const api = createApi({
  reducerPath: 'api',
  baseQuery: fetchBaseQuery({ baseUrl: 'https://api.example.com' }),
  tagTypes: ['User'],
  endpoints: (builder) => ({
    getUsers: builder.query<User[], void>({
      query: () => '/users',
      providesTags: ['User'],
    }),
    createUser: builder.mutation<User, Partial<User>>({
      query: (body) => ({ url: '/users', method: 'POST', body }),
      invalidatesTags: ['User'], // Invalide le cache après mutation
    }),
  }),
});

export const { useGetUsersQuery, useCreateUserMutation } = api;
```



Zustand — Alternative légère

```
import { create } from 'zustand';
import { persist, createJSONStorage } from 'zustand/middleware';
import AsyncStorage from '@react-native-async-storage/async-storage';

interface AuthStore {
  token: string | null;
  user: User | null;
  login: (token: string, user: User) => void;
  logout: () => void;
}

const useAuthStore = create<AuthStore>()(
  persist(
    (set) => ({
      token: null,
      user: null,
      login: (token, user) => set({ token, user }),
      logout: () => set({ token: null, user: null }),
    }),
    {
      name: 'auth-store',
      storage: createJSONStorage(() => AsyncStorage),
    }
  )
);
```



TanStack Query — State serveur

```
import { QueryClient, QueryClientProvider, useQuery, useMutation } from '@tanstack/react-query';

// Provider dans App.tsx
const queryClient = new QueryClient({
  defaultOptions: { queries: { staleTime: 5 * 60 * 1000 } },
});
<QueryClientProvider client={queryClient}><App /></QueryClientProvider>

// Dans un composant
function ListeFilms() {
  const { data, isLoading, error, refetch } = useQuery({
    queryKey: ['films'],
    queryFn: () => fetch('/api/films').then(r => r.json()),
  });

  if (isLoading) return <ActivityIndicator />;
  if (error) return <Text>Erreur !</Text>;
  return <FlatList data={data} renderItem={...} onRefresh={refetch} />;
}
```

Quelle solution choisir en 2025 ?

Besoin	Solution recommandée
State local simple	<code>useState</code> / <code>useReducer</code>
State global léger	Zustand
State atomique interdépendant	Jotai
Données serveur (fetch/cache)	TanStack Query
Grande app entreprise avec API	Redux Toolkit + RTK Query
State global + navigation	Zustand + Expo Router

*Combinaison populaire 2025 : **Zustand** (state local) + **TanStack Query** (state serveur)*

Module 4 · Construire son Interface

Composants, Flexbox, Navigation & Animations

View, Text, Image — Fondamentaux

```
import { View, Text, Image, StyleSheet, SafeAreaView } from 'react-native';
import { Image as ExpoImage } from 'expo-image'; // Recommandé en 2025

// View : conteneur (équivalent <div>)
<View style={styles.container}>

  /* Text : tout texte doit être dans <Text> */
  <Text style={styles.titre} numberOfLines={2} ellipsizeMode="tail">
    Titre tronqué après 2 lignes...
  </Text>

  /* Image locale */
  <Image source={require('./assets/logo.png')} style={{ width: 100, height: 100 }} />

  /* Image réseau – Expo Image pour cache et transitions */
  <ExpoImage source="https://example.com/photo.jpg"
    style={{ width: 200, height: 200 }}
    contentFit="cover" transition={300} />
</View>
```

Interactions & Gestes

```
import { TouchableOpacity, Pressable } from 'react-native';
import { Gesture, GestureDetector } from 'react-native-gesture-handler';

// TouchableOpacity – le plus courant
<TouchableOpacity onPress={handler} activeOpacity={0.7}>
  <Text>Appuyer</Text>
</TouchableOpacity>

// Pressable – contrôle fin de l'état "pressed"
<Pressable style={({ pressed }) => [styles.btn, pressed && styles.pressed]}
  onPress={handler} onLongPress={handleLong}>
  {({ pressed }) => <Text>{pressed ? 'Appuyé' : 'Normal'}</Text>}
</Pressable>

// Gesture Handler – gestes complexes (pan, swipe, pinch)
const swipe = Gesture.Pan().onUpdate(e => { x.value = e.translationX; });
<GestureDetector gesture={swipe}><Animated.View /></GestureDetector>
```



FlatList — Listes performantes

```
// FlatList = rendu virtuel – seuls les éléments visibles sont montés
<FlatList
  data={items}
  keyExtractor={(item) => item.id}
  renderItem={({ item, index }) => <ItemRow item={item} />}

  // Performance – tailles fixes = scroll fluide garanti
  getItemLayout={(_, index) => ({
    length: ITEM_HEIGHT,
    offset: ITEM_HEIGHT * index,
    index,
  })}

  // UX
  ListEmptyComponent={<Text>Aucun résultat</Text>}
  ListHeaderComponent={<Header />}
  ListFooterComponent={isLoading ? <ActivityIndicator /> : null}
  onEndReached={chargerPageSuivante}
  onEndReachedThreshold={0.3}
  numColumns={2}    // Grille 2 colonnes
/>
```



ScrollView & SectionList

```
// ScrollView : pour contenu court – rend TOUT en mémoire
<ScrollView
  showsVerticalScrollIndicator={false}
  contentContainerStyle={{ padding: 16 }}
  refreshControl={
    <RefreshControl refreshing={refreshing} onRefresh={onRefresh} />
  }
>
  {/* Contenu varié – formulaires, pages de détail */}
</ScrollView>

// SectionList : FlatList avec groupes/sections
<SectionList
  sections={[
    { title: 'Fruits', data: ['Pomme', 'Banane', 'Cerise'] },
    { title: 'Légumes', data: ['Carotte', 'Brocoli'] },
  ]}
  renderItem={({ item }) => <Text>{item}</Text>}
  renderSectionHeader={({ section }) => <Text style={styles.header}>{section.title}</Text>}
  stickySectionHeadersEnabled
/>
```

Flexbox en React Native

```
// Différences clés vs CSS web :  
// → flexDirection par défaut : 'column' (pas 'row')  
// → alignContent par défaut : 'flex-start'  
// → flexShrink par défaut : 0  
  
<View style={{  
  flex: 1,                // Prend tout l'espace dispo  
  flexDirection: 'row',   // row | column | row-reverse | column-reverse  
  justifyContent: 'space-between', // Axe principal  
  alignItems: 'center',  // Axe secondaire  
  flexWrap: 'wrap',      // Retour à la ligne  
  gap: 16,               // Espacement uniforme (RN 0.71+)  
}}>  
  <View style={{ flex: 1 }} />    {/* Proportion flexible */}  
  <View style={{ width: 80 }} />  {/* Taille fixe */}  
</View>
```

Patterns de layout courants

```
// Header fixe + contenu flexible + footer fixe
<View style={{ flex: 1 }}>
  <View style={{ height: 56, backgroundColor: '#0d47a1' }}>
    <Text style={{ color: '#fff', padding: 16 }}>Header</Text>
  </View>
  <ScrollView style={{ flex: 1 }}>{/* Contenu */}</ScrollView>
  <View style={{ height: 60, borderTopWidth: 1 }}>
    <Text style={{ textAlign: 'center', padding: 18 }}>Footer</Text>
  </View>
</View>

// Responsive : useWindowDimensions
const { width } = useWindowDimensions();
const cols = width > 600 ? 3 : 2;
<FlatList numColumns={cols} ... />
```

StyleSheet & NativeWind

```
// StyleSheet.create – valide les styles à la compilation
const styles = StyleSheet.create({
  card: {
    backgroundColor: '#fff',
    borderRadius: 12,
    padding: 16,
    // Ombres iOS
    shadowColor: '#000',
    shadowOffset: { width: 0, height: 2 },
    shadowOpacity: 0.12,
    shadowRadius: 4,
    // Ombre Android
    elevation: 4,
  },
});

// NativeWind 4 – Tailwind CSS en React Native
import { View, Text } from 'react-native';
<View className="bg-white rounded-2xl p-4 shadow-md mx-4 my-2">
  <Text className="text-lg font-bold text-blue-900">Titre</Text>
</View>
```


Mode sombre & thème dynamique

```
import { useColorScheme } from 'react-native';

// Détecter le thème système
function MonComposant() {
  const scheme = useColorScheme(); // 'light' | 'dark' | null

  const styles = StyleSheet.create({
    container: {
      flex: 1,
      backgroundColor: scheme === 'dark' ? '#1a1a2e' : '#ffffff',
    },
    texte: {
      color: scheme === 'dark' ? '#e0e0e0' : '#1a1a2e',
    },
  });

  return (
    <View style={styles.container}>
      <Text style={styles.texte}>Adapté au thème système</Text>
    </View>
  );
}
```

Navigation — Vue d'ensemble 2025

Deux solutions principales, toutes deux basées sur `@react-navigation/native` :

	Expo Router	React Navigation 7
Routing	File-based (comme Next.js)	Composants déclaratifs
Setup	Intégré à Expo	Configuration manuelle
Deep linking	Automatique	Config nécessaire
Typage	TypeScript natif	Typage manuel
Recommandé	Nouveaux projets Expo	Contrôle maximal

Expo Router — File-Based Routing

```
app/
├── _layout.tsx      → Layout racine (configurer tabs, drawer...)
├── index.tsx        → Route "/"
├── profil.tsx       → Route "/profil"
├── [id].tsx         → Route dynamique "/film/42"
├── (auth)/          → Groupe – invisible dans l'URL
│   ├── connexion.tsx → Route "/connexion"
│   └── inscription.tsx → Route "/inscription"
├── (tabs)/          → Groupe avec onglets
│   ├── _layout.tsx  → Configuration des tabs
│   ├── index.tsx    → Onglet Accueil
│   └── explorer.tsx → Onglet Explorer
└── +not-found.tsx  → Page 404
```



Expo Router — Naviguer

```
import { Link, useRouter, useLocalSearchParams } from 'expo-router';

// Lien déclaratif
<Link href="/profil">Voir le profil</Link>
<Link href={{ pathname: '/film/[id]', params: { id: '42' } }}>Film</Link>

// Navigation impérative
const router = useRouter();
router.push('/profil');
router.replace('/accueil'); // Sans retour possible
router.back();
router.navigate('/detail'); // Intelligent : pop si déjà dans la pile

// Récupérer les paramètres
const { id } = useLocalSearchParams<{ id: string }>();
```



React Navigation 7 — Stack & Tabs

```
import { createNativeStackNavigator } from '@react-navigation/native-stack';
import { createBottomTabNavigator } from '@react-navigation/bottom-tabs';

// Stack Navigator
const Stack = createNativeStackNavigator();
<Stack.Navigator>
  <Stack.Screen name="Accueil" component={AccueilScreen}
    options={{ headerShown: false }} />
  <Stack.Screen name="Detail" component={DetailScreen}
    options={{ title: 'Détail', presentation: 'modal' }} />
</Stack.Navigator>

// Tab Navigator
const Tab = createBottomTabNavigator();
<Tab.Navigator screenOptions={{ tabBarActiveTintColor: '#0d47a1' }}>
  <Tab.Screen name="Accueil" component={AccueilScreen}
    options={{ tabBarIcon: ({ color }) => <Icon name="home" color={color} /> }} />
</Tab.Navigator>
```



Bibliothèques communauté incontournables

React Native Paper

Material Design 3 — Cards, FAB, DataTable, Dialog, Snackbar

Expo Image

Cache avancé, transitions, blurhash placeholder, SVG, contentFit

React Native Reanimated 3

Animations worklet 60fps, gestes, layout animations, shared elements

React Native Skia

Dessins vectoriels GPU, filtres, effets, graphiques — basé sur Skia



Animated API — Fondamentaux

```
import { Animated, Easing } from 'react-native';

function FadeIn({ children }) {
  const opacity = useRef(new Animated.Value(0)).current;

  useEffect(() => {
    Animated.timing(opacity, {
      toValue: 1,
      duration: 500,
      easing: Easing.out(Easing.cubic),
      useNativeDriver: true, // ⚡ Obligatoire pour opacity & transform
    }).start();
  }, []);

  return <Animated.View style={{ opacity }}>{children}</Animated.View>;
}

// Combinaisons d'animations
Animated.sequence([fadeIn, slideUp]).start();
Animated.parallel([fadeIn, scaleUp]).start();
Animated.stagger(100, items.map(makeAnim)).start();
```

⚡ Reanimated 3 — Animations performantes

```
import Animated, {
  useSharedValue, useAnimatedStyle,
  withSpring, withTiming, withRepeat,
} from 'react-native-reanimated';
import { Gesture, GestureDetector } from 'react-native-gesture-handler';

function CarteInteractive() {
  const scale = useSharedValue(1);

  const style = useAnimatedStyle(() => ({
    transform: [{ scale: scale.value }],
  }));

  const tap = Gesture.Tap()
    .onBegin(() => { scale.value = withSpring(0.95); })
    .onFinalize(() => { scale.value = withSpring(1); });

  return (
    <GestureDetector gesture={tap}>
      <Animated.View style={[styles.carte, style]} />
    </GestureDetector>
  );
}
```




LayoutAnimation & Moti

```
// LayoutAnimation – transitions automatiques du layout
import { LayoutAnimation, UIManager, Platform } from 'react-native';
if (Platform.OS === 'android')
  UIManager.setLayoutAnimationEnabledExperimental(true);

const supprimer = (id) => {
  LayoutAnimation.configureNext(LayoutAnimation.Presets.spring);
  setItems(prev => prev.filter(i => i.id !== id));
};

// Moti – animations déclaratives simples
import { MotiView } from 'moti';
<MotiView
  from={{ opacity: 0, translateY: -20 }}
  animate={{ opacity: 1, translateY: 0 }}
  transition={{ type: 'timing', duration: 400, delay: 100 }}
>
  {/* Contenu animé */}
</MotiView>
```

TP — Jour 2

Objectif : Initialiser un projet complet et développer une première application



TP — Application "FilmBrowser"

Architecture cible :

```
app/
├── _layout.tsx      → Tabs : Accueil + Favoris
├── (tabs)/
│   ├── index.tsx   → Liste des films populaires
│   └── favoris.tsx → Films ajoutés aux favoris
└── film/[id].tsx   → Détail d'un film

store/
└── favorisSlice.ts → Redux Toolkit – gestion favoris

components/
└── FilmCard.tsx    → Composant carte film réutilisable
```



TP — Composant FilmCard

```
// components/FilmCard.tsx
import { MotiView } from 'moti';
import { Image } from 'expo-image';
import { Link } from 'expo-router';

const POSTER_BASE = 'https://image.tmdb.org/t/p/w300';

const FilmCard = ({ film, index }: { film: Film; index: number }) => (
  <MotiView
    from={{ opacity: 0, translateY: 20 }}
    animate={{ opacity: 1, translateY: 0 }}
    transition={{ delay: index * 60 }}
  >
    <Link href={{ pathname: '/film/[id]', params: { id: film.id } }}>
      <View style={styles.card}>
        <Image source={POSTER_BASE + film.poster_path}
          style={styles.poster} contentFit="cover" />
        <Text style={styles.titre} numberOfLines={1}>{film.title}</Text>
        <Text style={styles.note}>★ {film.vote_average.toFixed(1)}</Text>
      </View>
    </Link>
  </MotiView>
);
```



TP — Écran liste avec TanStack Query

```
// app/(tabs)/index.tsx
import { useQuery } from '@tanstack/react-query';

const API_KEY = 'VOTRE_CLE_TMDB'; // Compte gratuit sur themoviedb.org

async function fetchFilmsPopulaires() {
  const r = await fetch(
    `https://api.themoviedb.org/3/movie/popular?api_key=${API_KEY}&language=fr-FR`
  );
  const data = await r.json();
  return data.results as Film[];
}

export default function AccueilScreen() {
  const { data: films, isLoading, refetch } = useQuery({
    queryKey: ['films', 'popular'],
    queryFn: fetchFilmsPopulaires,
  });

  if (isLoading) return <ActivityIndicator style={{ flex: 1 }} />;
  return (
    <FlatList data={films} numColumns={2}
      keyExtractor={f => f.id.toString()}
      renderItem={({ item, index }) => <FilmCard film={item} index={index} /> } />
  );
}
```



TP — Store Favoris (Redux Toolkit)

```
// store/favorisSlice.ts
import { createSlice, PayloadAction } from '@reduxjs/toolkit';

interface Film { id: number; title: string; poster_path: string; }

const favorisSlice = createSlice({
  name: 'favoris',
  initialState: [] as Film[],
  reducers: {
    ajouterFavori: (state, action: PayloadAction<Film>) => {
      if (!state.find(f => f.id === action.payload.id))
        state.push(action.payload);
    },
    retirerFavori: (state, action: PayloadAction<number>) => {
      return state.filter(f => f.id !== action.payload);
    },
  },
});

export const { ajouterFavori, retirerFavori } = favorisSlice.actions;

// Sélecteur
export const selectEstFavori = (id: number) =>
  (state: RootState) => state.favoris.some(f => f.id === id);
```

? Questions de révision — Jour 2

- Quelle est la différence entre Props et State ?
- En quoi le pattern Flux se distingue-t-il du MVC ?
- Que fait `createSlice` dans Redux Toolkit ? Pourquoi Immer est-il intégré ?
- Quelle est la différence entre `FlatList` et `ScrollView` ? Quand utiliser laquelle ?
- Comment fonctionne le routing file-based d'Expo Router ?
- Pourquoi faut-il utiliser `useNativeDriver: true` dans les animations ?

